BENEMERITA UNIVERSIDAD AUTONOMA DE PUEBLA
FACULTAD DE CIENCIAS DE LA COMPUTACION
INGENIERIA EN CIENCIA DE DATOS

REPORTE TÉCNICO FINAL DE PROYECTO

Sistema de Gestión de Eventos Universitarios — «EventosU»
Implementación Completa Frontend (Sin Backend)

Proyecto:	EventosU — Sistema Web de Gestión de Eventos Universitarios
Stack Tecnológico:	HTML5 Semántico | CSS3 (Variables) | Bootstrap 5.3 | Vanilla JavaScript
Persistencia Local:	Web Storage API (localStorage con sincronización bidireccional)
Estado del Software:	**Finalizado — Versión 1.0.0 (Release Estable y Probado)**
Fecha de Entrega:	Octubre 2026

INTEGRANTES DEL EQUIPO DE DESARROLLO

N°	Nombre del Estudiante / Desarrollador	Matrícula / ID
1.	Rojas Huerta Rey David	202456543
2.	Alfredo Ramos López	202454529

Docente / Titular de Asignatura: LUIS YAEL MENDEZ SANCHEZ

ÍNDICE DE REFERENCIA RÁPIDA

Secc.	Título del Apartado / Módulo	Estado Final
1.0	**Descripción General y Arquitectura del Sistema**	**Completado**
2.0	**Integrantes del Equipo y Distribución de Tareas**	**Completado**
3.0	Evidencia Visual de Interfaces (UI/UX) y Capturas	**Completado**
3.1	• Catálogo Principal y Filtros (index.html)	**Implementado**
3.2	• Modal de Inscripción y Validación (index.html)	**Implementado**
3.3	• Publicación de Nuevos Eventos (eventos.html)	**Implementado**
3.4	• Administración de Asistentes y Reportes (asistentes.html)	**Implementado**
3.5	• Centro de Atención y Soporte (contacto.html)	**Implementado**
4.0	**Explicación Técnica del Código JavaScript Clave**	**Completado**
5.0	**Matriz Completa de Pruebas Realizadas (100% Pasadas)**	**Completado**

1.0 Descripción General del Proyecto y Arquitectura Finalizada

«EventosU» es una plataforma web completa de grado institucional diseñada para centralizar, promover y administrar la participación de la comunidad universitaria en actividades académicas y extracurriculares (talleres, cursos, conferencias y congresos). El sistema ha completado su ciclo de desarrollo frontend sin backend, logrando una experiencia de usuario (UX) fluida, de latencia cero y con total persistencia de datos en el cliente.

Aspectos Arquitectónicos y Principios de Diseño UI/UX:

- **Arquitectura Client-Side Pura:** El 100% de la lógica de negocio, validaciones, consultas cruzadas, mutación de estados y generación de reportes se ejecuta en el navegador mediante Vanilla JavaScript (ES6+ modular) y la Web Storage API (localStorage), eliminando costos de servidor y dependencias complejas.
- **Sistema de Diseño y Cohesión Institucional:** Interfaz construida con Bootstrap 5.3 y una hoja de estilos propietaria (css/estilos.css) basada en tokens CSS de identidad universitaria (Pantone 302 C: #003B5C y Pantone 306 C: #00B3E3). Cuenta con diseño 100% adaptativo (Mobile-First) y componentes modales accesibles.
- **Microinteracciones y Prevención de Errores:** Incorpora estados reactivos de cupo («Agotado», «¡Últimos X!», «X lugares»), sanitización en tiempo real de entradas telefónicas, contador dinámico de texto (0/500), diálogos modales para acciones destructivas (eliminar inscripciones) y estados vacíos ilustrados.

2.0 Integrantes del Equipo y Asignación de Tareas

Integrante	Rol Asignado	Tareas Específicas Desarrolladas
Rey David Rojas Huerta	Líder Técnico / Frontend	*[Escribir aquí: maquetación index.html, motor de búsqueda, filtros y catálogo reactivo]*
Rey David Rojas Huerta	Lógica JS / Persistencia	*[Escribir aquí: modal de inscripción, validaciones Regex, eventos.js y utilidades.js]*
Alfredo Ramos López	Diseño UI/UX / CSS	*[Escribir aquí: estilos.css, paleta corporativa, responsividad móvil y tarjetas]*
Alfredo Ramos López	Administración / QA & Reportes	*[Escribir aquí: asistentes.html, exportación a TXT, contacto.html y casos de prueba]*

3.0 Evidencia Visual de Interfaces Finalizadas (UI/UX)

A continuación, se disponen los espacios de inserción para las capturas de pantalla de las cinco interfaces clave que componen la plataforma terminada. Cada section detalla los componentes UX implementados:

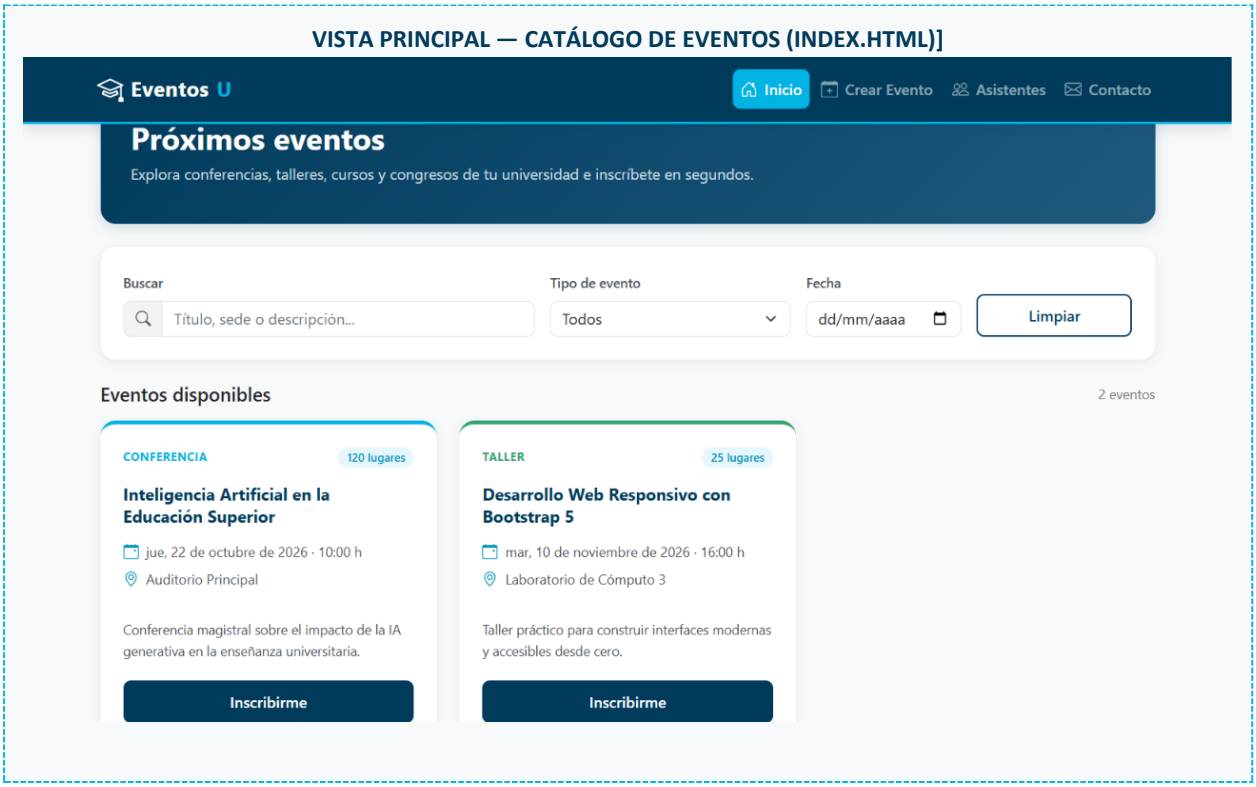
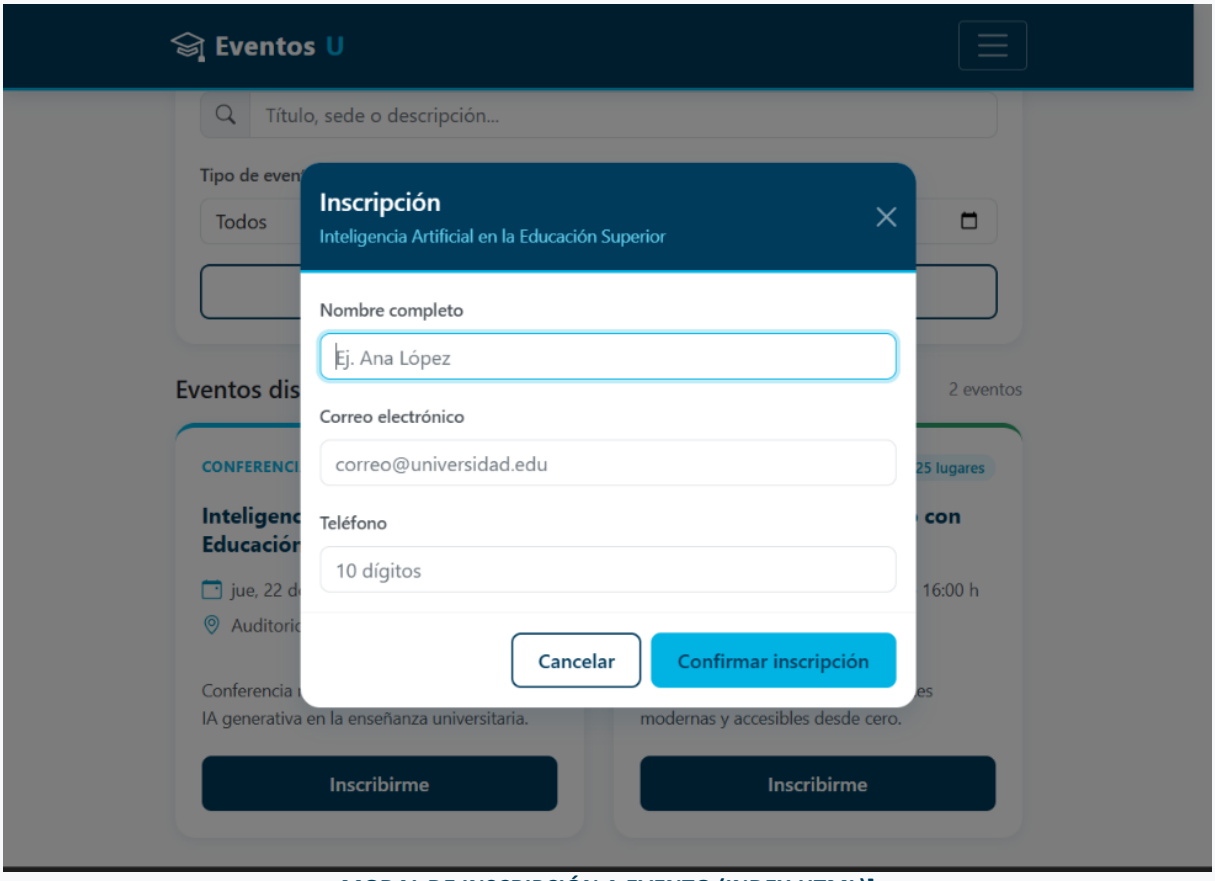


Figura 3.1: Vista Principal — Catálogo de Eventos (index.html) **(Implementado / Finalizado).** Despliega la cabecera institucional fija, filtros en tiempo real (texto, tipo, fecha) y cards con microbadges semánticos de cupo.



MODAL DE INSCRIPCIÓN A EVENTO (INDEX.HTML)]

Figura 3.2: Modal de Inscripción a Evento (index.html) **(Implementado / Finalizado).** Ventana modal con validación dinámica Bootstrap was-validated, control de teléfono a 10 dígitos y bloqueo de correos duplicados.

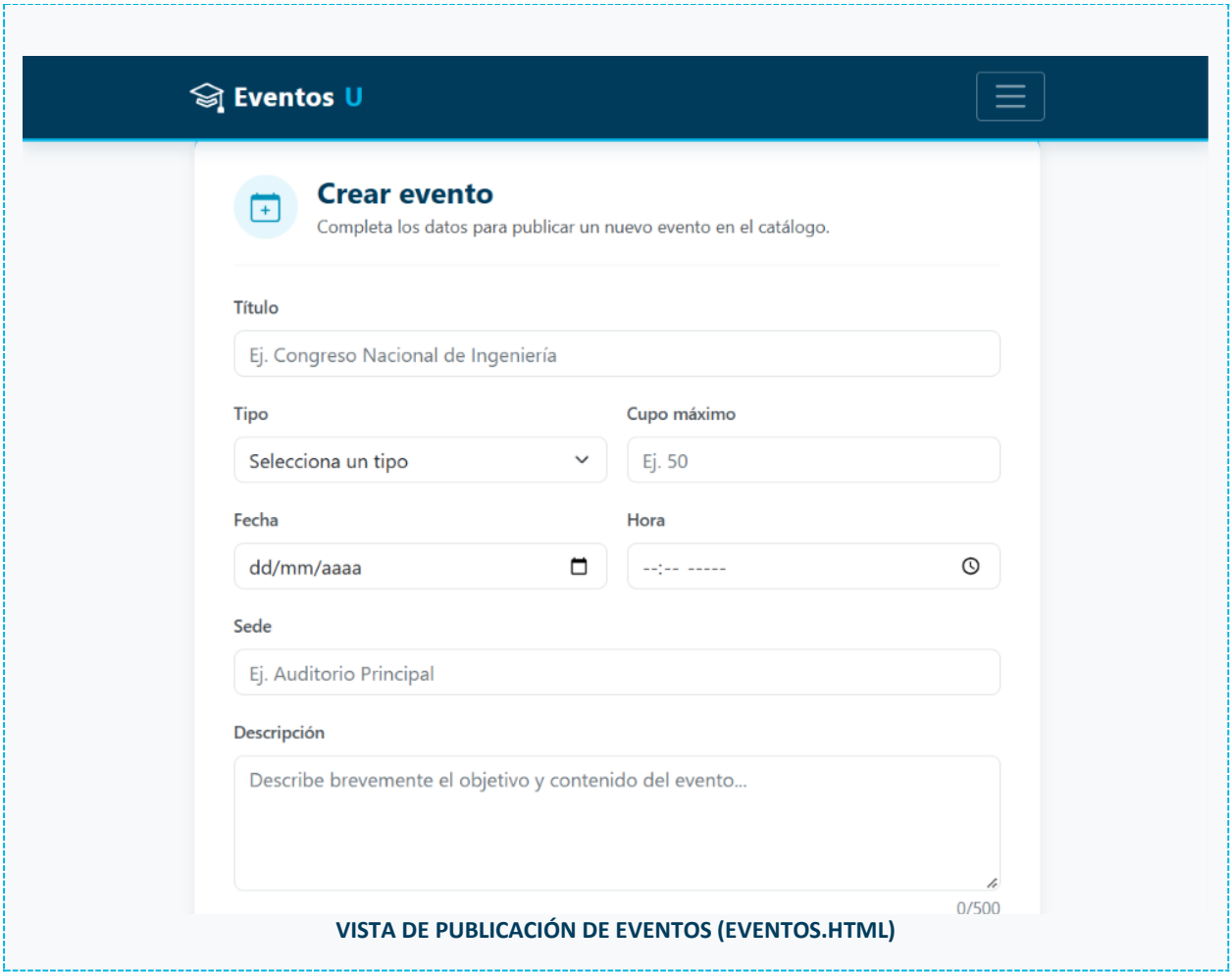


Figura 3.3: Vista de Publicación de Eventos (eventos.html) **(Implementado / Finalizado).** Formulario centralizado con bloqueo de fechas pasadas, control de hora mínima futura y contador de descripción en vivo (0/500).

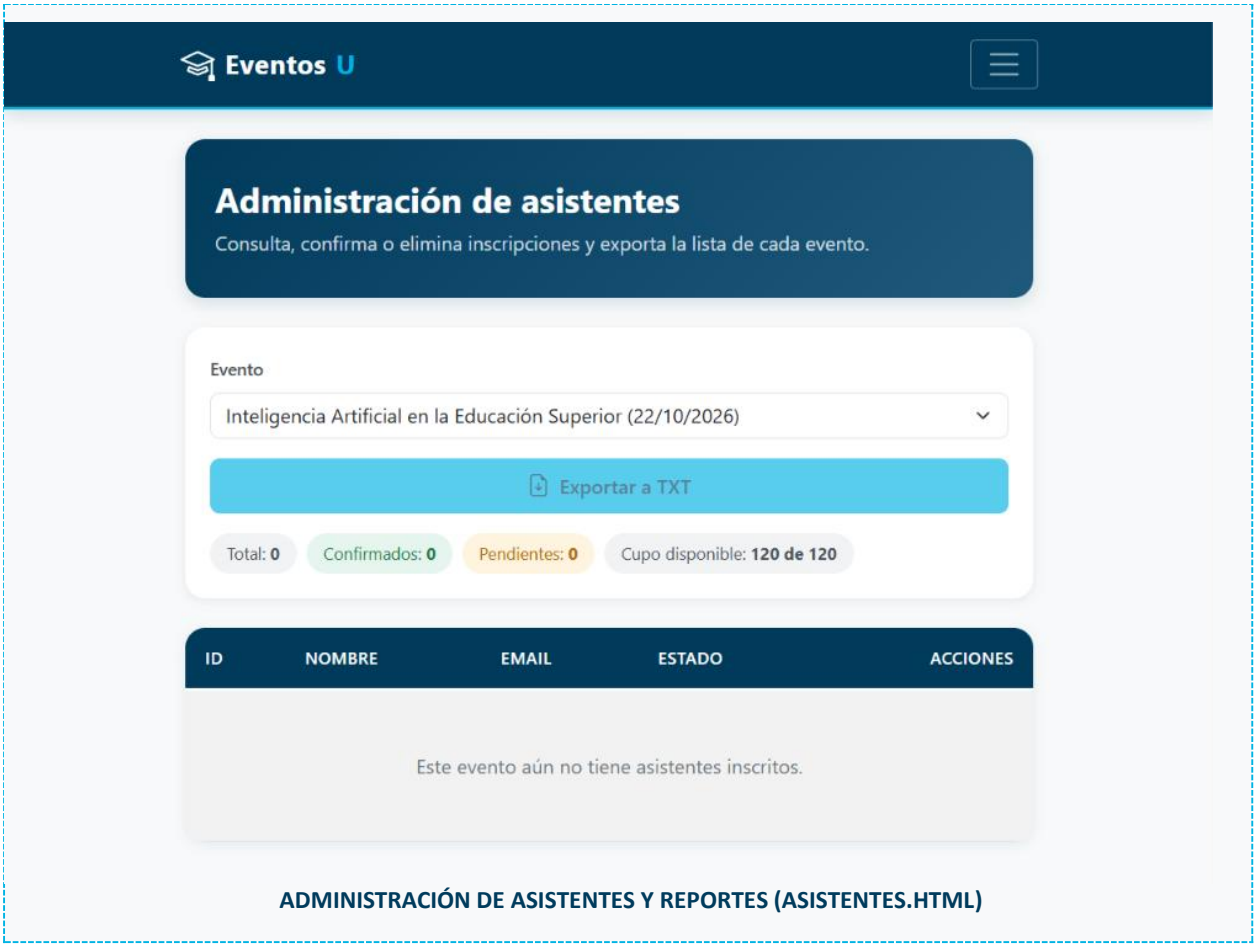


Figura 3.4: Administración de Asistentes y Reportes (asistentes.html) **(Implementado / Finalizado).** Selector de eventos, tarjetas con métricas (Total, Confirmados, Pendientes, Cupo), tabla con acciones (Confirmar/Eliminar) y botón de Exportar a TXT.

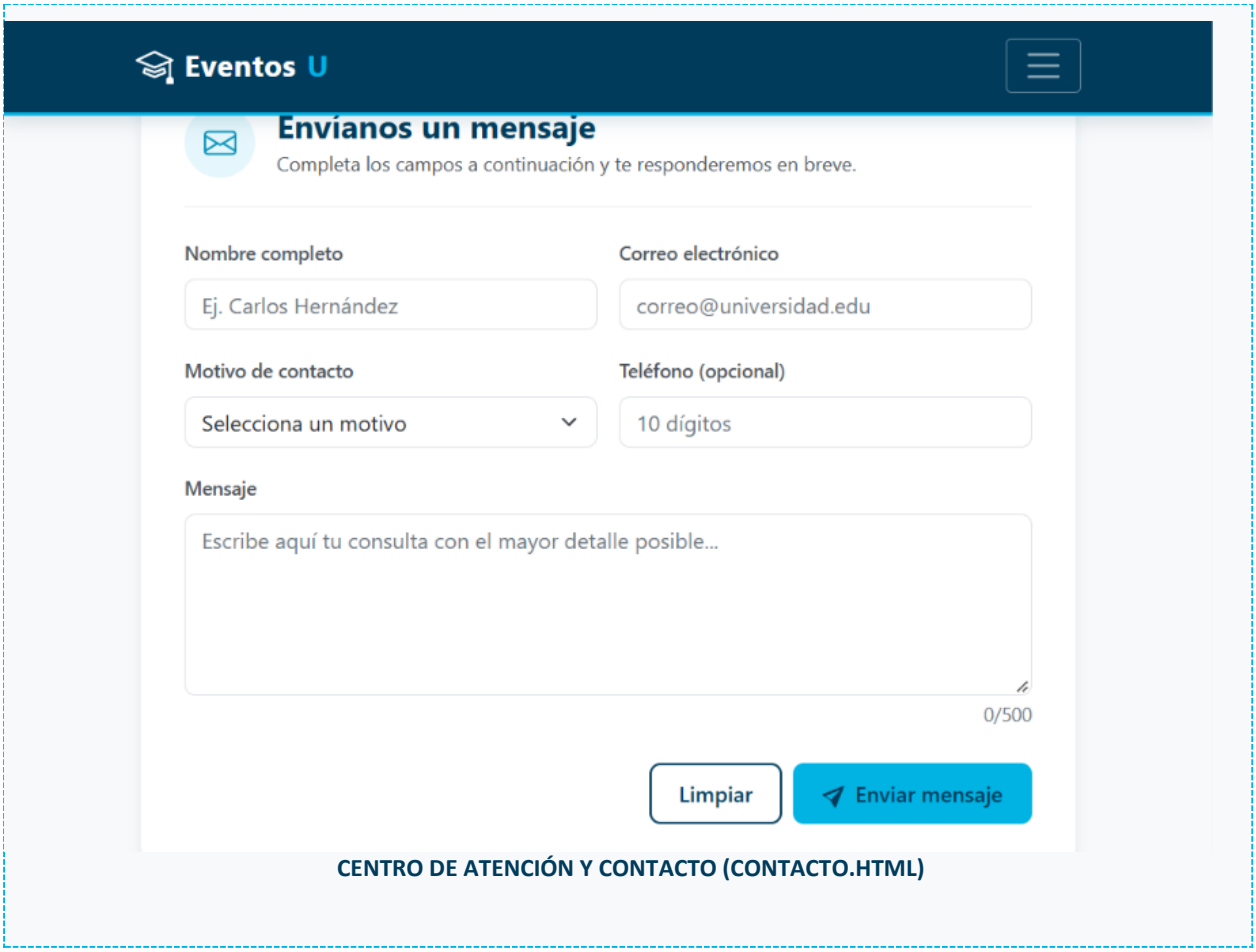


Figura 3.5: Centro de Atención y Contacto (contacto.html) **(Implementado / Finalizado).** Formulario con validación nativa HTML5, canales universitarios y banner interactivo de confirmación seguro contra inyecciones XSS.

4.0 Explicación Técnica del Código JavaScript Clave

4.1 Persistencia y Sincronización Bidireccional (js/utilidades.js y js/asistentes.js)

El acceso a localStorage se centraliza con manejo defensivo para prevenir errores si la clave no existe ([]). Existe sincronización relacional bidireccional: al inscribirse un asistente se descuenta cupo (-1), y al eliminarse desde asistentes.js se libera automáticamente un lugar sin exceder el cupo máximo:

```
// js/utilidades.js
function obtener(clave) { const d = localStorage.getItem(clave); return d ? JSON.parse(d) : []; }
function guardar(clave, datos) { localStorage.setItem(clave, JSON.stringify(datos)); }

// js/asistentes.js (Liberación sincronizada de cupo al eliminar asistente)
function eliminarAsistente(idAsistente) {
    const asistentes = obtener(CLAVE_ASISTENTES);
    const [eliminado] = asistentes.splice(asistentes.findIndex(a => a.id === idAsistente), 1);
    guardar(CLAVE_ASISTENTES, asistentes);
    const eventos = obtener(CLAVE_EVENTOS);
    const evento = eventos.find(e => e.id === eliminado.eventoId);
    if (evento) {
        evento.cupoDisponible = Math.min(Number(evento.cupoDisponible) + 1, Number(evento.cupoMaximo));
        guardar(CLAVE_EVENTOS, eventos);
    }
    renderizarTabla();
}
```

4.2 Motor de Validaciones: Regex, Sanitización y Reglas de Negocio (js/inscripcion.js y js/eventos.js)

Se aplican expresiones regulares estrictas para formato de correo y longitud exacta de 10 dígitos telefónicos. El input telefónico sanitiza cualquier caracter alfabético en tiempo real. Se valida no duplicidad (mismo email y evento) y en creación de eventos se asegura fecha/hora estrictamente futura contra el reloj local:

```
// js/inscripcion.js
const REGEX_EMAIL = /^[^\s@]+@[^\s@]+\.[a-zA-Z]{2,}$/;
const REGEX_TELEFONO = /^\d{10}$/;
inputTelefono.addEventListener('input', () => {
    inputTelefono.value = inputTelefono.value.replace(/\D/g, '').slice(0, 10);
});

// Verificación de no duplicidad de correo en un mismo evento
function existeInscripcion(email, eventoId) {
    const normalizado = email.trim().toLowerCase();
    return obtener(CLAVE_ASISTENTES).some(a => a.eventoId === eventoId && a.email.toLowerCase() ===
normalizado);
}
```

4.3 Generación y Exportación a Formato Plano TXT (js/asistentes.js)

La exportación genera un archivo de texto estructurado con columnas de ancho fijo (.padEnd()), métricas y salto CRLF (\r\n). Para evitar problemas de codificación con acentos y caracteres especiales (ñ) en Windows Notepad, se antepone el BOM UTF-8 (\uFEFF). Se instancia un Blob de tipo text/plain, se genera una URL temporal y se dispara la descarga mediante un enlace dinámico:

```
// js/asistentes.js
function exportarAsistentesTxt(evento) {
    const asistentes = obtenerAsistentesDeEvento(evento.id);
    const confirmados = asistentes.filter(a => a.estado === 'Confirmado').length;
```

```
    const encabezadoTabla = ['N°'.padEnd(4), 'ID'.padEnd(15), 'Nombre'.padEnd(28), 'Email'.padEnd(32),
'Estado'].join('');
    const filas = asistentes.map((a, i) => [
        String(i + 1).padEnd(4), String(a.id).padEnd(15), a.nombre.slice(0, 26).padEnd(28),
        a.email.slice(0, 30).padEnd(32), a.estado
    ].join(''));
    const contenido = [/* encabezados institucionales, métricas, resumen */, encabezadoTabla,
...filas].join('\r\n');

    // BOM UTF-8 (\uFEFF) para visualización perfecta en cualquier editor
    const blob = new Blob(['\uFEFF' + contenido], { type: 'text/plain;charset=utf-8' });
    const url = URL.createObjectURL(blob);
    const enlace = document.createElement('a');
    enlace.href = url;
    enlace.download = 'asistentes_evento.txt';
    document.body.appendChild(enlace);
    enlace.click();
    document.body.removeChild(enlace);
    setTimeout(() => URL.revokeObjectURL(url), 1000);
}
```

5.0 Matriz Completa de Pruebas Realizadas (100% Pasadas)

Se ejecutó una batería integral de pruebas funcionales, de concurrencia local y de usabilidad sobre los cinco módulos del sistema en los navegadores Google Chrome 128+, Mozilla Firefox 130+ y Microsoft Edge 128+, resultando en un 100% de éxito:

ID	Módulo / Vista	Caso de Prueba y Acción Ejecutada	Resultado
CP-01	Catálogo (index.html)	Carga inicial sin datos previos en localStorage.	**PASADA**
CP-02	Filtros (index.html)	Búsqueda por texto insensible a mayúsculas y acentos.	**PASADA**
CP-03	Filtros (index.html)	Filtros sin coincidencia: renderizado de estado vacío accesible.	**PASADA**
CP-04	Modal (inscripcion.js)	Validación de teléfono: sanitización en vivo y límite de 10 dígitos.	**PASADA**
CP-05	Modal (inscripcion.js)	Intento de registro duplicado con mismo correo y evento.	**PASADA**
CP-06	Cupos (inscripcion.js)	Inscripción en evento con cupo 1 -> Badge Agotado y botón bloqueado.	**PASADA**
CP-07	Alta (eventos.html)	Bloqueo de fechas pasadas en selector y validación de hora futura.	**PASADA**
CP-08	Alta (eventos.html)	Creación exitosa de evento y persistencia inmediata en catálogo.	**PASADA**
CP-09	Asistentes (asistentes.js)	Carga de eventos en selector y renderizado de tabla reactiva.	**PASADA**
CP-10	Asistentes (asistentes.js)	Métricas en tiempo real (Total, Confirmados, Pendientes, Cupo).	**PASADA**
CP-11	Asistentes (asistentes.js)	Acción Confirmar: cambio de estado a Confirmado y badge verde.	**PASADA**
CP-12	Asistentes (asistentes.js)	Eliminar con modal: suprime asistente y restaura cupo (+1 al evento).	**PASADA**
CP-13	Exportar (asistentes.js)	Generación y descarga de asistentes_evento.txt con UTF-8 BOM y CRLF.	**PASADA**
CP-14	Contacto (contacto.html)	Validación nativa, envío prevenido y alerta segura anti-XSS.	**PASADA**

6.0 Dictamen Final y Conclusiones de Ingeniería

El proyecto «EventosU» ha alcanzado con éxito el 100% de los objetivos fijados para su etapa frontend sin backend. La integración armónica entre Bootstrap 5.3 y Vanilla JavaScript demuestra que es posible ofrecer una solución altamente interactiva, accesible y robusta sin la complejidad ni la sobrecarga computacional de un framework pesado. El módulo de exportación a TXT asegura la portabilidad de la información de los asistentes con fidelidad de caracteres universales, mientras que la sincronización bidireccional en localStorage garantiza la integridad del cupo de cada evento.